UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

# Sesión 18

## Taller 2 — Resolución de problemas

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

# Índice

**1 Instrucciones del taller** **2**

**2 Problema 1: El profesor y los nombres** **3**
2.1 Enunciado . . . . . 3
2.2 Parte 1 — Revisión guiada . . . . . 3
2.3 Parte 2 — Trabajo en equipo . . . . . 4
2.3.1 Análisis . . . . . 4
2.3.2 Pseudocódigo . . . . . 4

**3 Problema 2: Once Cifras** **5**
3.1 Enunciado . . . . . 5
3.2 Parte 1 — Revisión guiada . . . . . 5
3.3 Parte 2 — Trabajo en equipo . . . . . 5
3.3.1 Análisis . . . . . 5
3.3.2 Pseudocódigo . . . . . 6

**4 Resumen de la sesión** **7**

# Instrucciones del taller

Esta sesión sigue la misma dinámica que el Taller 1: trabajo en **equipos de dos o tres personas**, con dos momentos por problema.

**Parte 1 — Revisión guiada.** El profesor trabaja el caso de prueba en el pizarrón. Tu tarea es **poner atención**: observa cómo se leen los datos, qué operaciones se hacen y cómo se obtiene la salida esperada. No abras el cuaderno todavía.

**Parte 2 — Trabajo en equipo.** Con lo observado, tu equipo generaliza el algoritmo, lo implementa en C++ y lo envía al juez.

Durante la **Parte 1** no escribas código ni intentes adelantarte a la solución. El objetivo es entender *qué* pide el problema, no todavía *cómo* resolverlo para cualquier entrada.

**Flujo de la Parte 2:**

1. Con el ejemplo ya resuelto, identifica la condición o proceso general.
2. Escribe el algoritmo en pseudocódigo antes de escribir código.
3. Implementa en C++ y prueba con el mismo ejemplo del enunciado.
4. Envía al juez. Si recibes `WA`, construye un contraejemplo antes de modificar el código al azar.

# Problema 1: El profesor y los nombres

## • Enunciado

Un profesor se sabe de memoria solo algunos nombres de sus alumnos. Cuando necesita llamar a un alumno cuyo nombre no conoce, lo hace refiriéndose al alumno conocido más cercano.

Dados los $n$ alumnos en fila (algunos con nombre conocido, otros como `?`), y $m$ preguntas de la forma "¿cómo llamarías al alumno en la posición $p$?", escribe el programa que responda cada pregunta.

**Reglas para nombrar:**

- Si la posición $p$ tiene nombre conocido: imprimir el nombre.
- Si el alumno conocido más cercano está a distancia $d$ y está a la **izquierda**: imprimir *d* veces "`derecha`" seguido de "`de Nombre`".
- Si está a la **derecha**: imprimir *d* veces "`izquierda`" seguido de "`de Nombre`".
- Si hay dos alumnos conocidos a la **misma distancia** (uno a cada lado): imprimir "`justo enmedio de NombreIzq y NombreDer`".

**Entrada.** Primera línea: $n$ ($1 \leq n \leq 100$). Segunda línea: $n$ nombres separados por espacios (`?` si se desconoce; cadena de máximo 3 letras si se conoce). Al menos un nombre no es `?`. Tercera línea: $m$ ($1 \leq m \leq 100$). Las siguientes $m$ líneas: una posición $p$ ($1 \leq p \leq n$) por línea.

**Salida.** Por cada consulta, una línea con la forma de llamar al alumno.

## • Parte 1 — Revisión guiada

**Pon atención al pizarrón.** El profesor va a trazar la fila, marcar los nombres conocidos y resolver cada consulta paso a paso. Observa cómo se determina el nombre conocido más cercano y cómo se construye la cadena de respuesta.

| Entrada | Salida |
|---|---|
| `10`<br>`A ? ? B ? ? ? C ? ?`<br>`4`<br>`3`<br>`8`<br>`6`<br>`10` | `izquierda de B`<br>`C`<br>`justo enmedio de B y C`<br>`derecha derecha de C` |

**Notas de lo observado en el pizarrón:**

## • Parte 2 — Trabajo en equipo

***Análisis***

¿Cómo encuentras el nombre conocido más cercano **a la izquierda** de la posición $p$?

¿Cómo encuentras el más cercano **a la derecha**?

¿Cuándo se imprime "`justo enmedio`"? ¿Cuándo sólo "`izquierda`" o "`derecha`"?

¿Cómo generas la cadena con $d$ repeticiones de "`derecha`" o "`izquierda`" en C++?

> ✔ **Recuerda**
>
> Las posiciones en el enunciado van de 1 a $n$. En un arreglo de C++ los índices van de 0 a $n-1$. Recuerda restar 1 al leer $p$.

***Pseudocódigo***

# Problema 2: Once Cifras

## • Enunciado

Se tiene un número de **11 cifras**. Se parte en **tres partes** (cada parte es una subcadena contigua de dígitos) y se suman. ¿Cuál es la menor suma posible de entre todas las formas de hacer el corte?

**Entrada.** Un número entero de 11 dígitos.

**Salida.** La menor suma posible.

## • Parte 1 — Revisión guiada

**Pon atención al pizarrón.** El profesor va a mostrar qué significa "partir en tres partes", cómo se calcula la suma de una partición concreta y cuántas particiones distintas hay en total. Observa el patrón antes de pensar en la solución general.

| Entrada | Salida |
|---|---|
| `31124510988` | `3750` |

**Notas de lo observado en el pizarrón:**

## • Parte 2 — Trabajo en equipo

***Análisis***

¿Cuántos puntos de corte necesitas para dividir una cadena de 11 dígitos en 3 partes?

Si la cadena tiene 11 dígitos y los dos puntos de corte están en las posiciones $i$ y $j$ (con $1 \leq i < j \leq 10$), ¿cuántas combinaciones $(i, j)$ hay?

¿Qué ocurre con los ceros a la izquierda al convertir una subcadena a número? (*Ejemplo: "0988" como entero es 988.*) ¿Eso afecta el resultado?

¿Cabe el valor máximo posible de la suma en un `int`? (*Pista: el peor caso es partir*

*"99999999999" en "9" + "9" + "999999999".*)

**✔ Recuerda**

Si lees el número como `string`, puedes usar `substr` para obtener cada parte y `stoll` para convertirla a entero. Iterar sobre todos los pares $(i, j)$ con dos `for` anidados es suficiente dado el tamaño del problema.

***Pseudocódigo***

## Resumen de la sesión

- Para “El profesor y los nombres”: busca el nombre conocido más cercano a la izquierda y a la derecha; la distancia determina cuántas veces se repite “`izquierda`” o “`derecha`”.
- Para “Once Cifras”: hay $\binom{10}{2} = 45$ formas de partir 11 dígitos en 3 partes; iterar sobre todas y quedarse con la suma mínima es la estrategia correcta.
- Los ceros iniciales en una subcadena no son un caso especial: `stoll("0988")` devuelve 988 automáticamente.